

Perche la Tua Azienda Ha Bisogno di un Team di Sviluppo Esterno

Hai un software che funziona. Ma è fermo. Le richieste si accumulano, nessuno le realizza. Il tuo sviluppatore storico ha lasciato. O hai un fornitore che "risponde quando può", senza processo, senza priorità, senza prevedibilità.

Questo manuale esamina le quattro opzioni che hai per far evolvere il tuo software nel tempo — sviluppatore interno, freelancer, software house, team factory in retainer — con i loro vantaggi reali, i loro rischi reali, e i segnali per capire quale fa per te. Senza cifre di riferimento che cambiano con il mercato, ma con i criteri qualitativi che restano validi indipendentemente dal momento.

1. Il problema di fondo: il software non è mai finito

Quando commissioniamo un software, tendiamo a pensarlo come un progetto con un inizio e una fine. Si costruisce, si consegna, si usa. Fatto.

Non funziona così.

Un software in produzione è un sistema vivo. Richiede aggiornamenti di sicurezza, adattamenti ai nuovi sistemi operativi e browser, correzioni di bug che emergono dall'uso reale, e soprattutto evoluzione: nuove funzionalità, nuovi processi da supportare, nuove esigenze degli utenti.

Le aziende che trattano il software come un progetto finito si trovano invariabilmente nella stessa situazione dopo qualche anno: un sistema che funzicchia, che nessuno sa come modificare senza rischi, che ha accumulato debito tecnico invisibile, e che è diventato un freno invece che uno strumento.

La domanda non è quindi "come costruisco il mio software?" — ma "chi fa evolvere il mio software nel tempo?"

Le risposte disponibili sono quattro, e ognuna ha caratteristiche precise.

*****Insight 108 Vision**** — Con oltre vent'anni di architettura enterprise, ho osservato che le aziende con i migliori prodotti digitali non sono quelle che hanno trovato lo sviluppatore più bravo. Sono quelle che hanno trovato il modello di relazione giusto — quello che allinea gli incentivi del fornitore con i risultati del cliente.*

2. Scenario A: Lo sviluppatore interno

La prima opzione che viene in mente è assumere. È la più naturale, la più percepita come "sicura", e spesso quella con il profilo di rischio più sottovalutato.

Cosa funziona

Un developer interno conosce il prodotto profondamente. Con il tempo diventa un esperto del dominio aziendale, non solo della tecnologia. Può rispondere rapidamente a richieste urgenti. Non ha overhead di comunicazione con un fornitore esterno. Per prodotti core che evolvono continuamente con alta urgenza, può essere la scelta giusta.

Cosa non funziona

Concentrazione del know-how. Quando un solo sviluppatore conosce un sistema, l'azienda è ostaggio di quella persona. Se va in ferie, se si ammala, se lascia, il sistema si ferma. Ho visto aziende bloccarsi per mesi dopo che lo

sviluppatore storico ha lasciato, perché nessun altro sapeva come funzionava il codice.

Assenza di peer review. Un developer solo non ha nessuno che rivede il suo codice. Nessuna seconda opinione sulle scelte architettureali. Nessun controllo sulla qualità. La qualità tende a degradarsi nel tempo senza che nessuno se ne accorga — finché non è tardi.

Costo fisso elevato. Assumere porta con sé un costo fisso mensile indipendentemente dal volume di lavoro. Ci sono mesi con molto da fare e mesi con poco. Il costo non cambia.

Il rischio dell'isolamento tecnico. Uno sviluppatore solo, senza confronto con altri professionisti, tende a replicare le stesse soluzioni che conosce. L'aggiornamento tecnologico è spontaneo e non garantito.

Quando ha senso

Ha senso quando il prodotto è davvero core — centrale al business, in evoluzione continua, con alto volume di lavoro ogni mese. Quando l'azienda è disposta a investire nel recruiting, nella formazione, nella gestione della persona. E quando si accetta che uno solo non è mai sufficiente: serve un team, non un singolo.

3. Scenario B: Il freelancer on-demand

Il freelancer è spesso la prima scelta delle PMI che non vogliono o non possono assumere. È flessibile, diretto, apparentemente economico.

Cosa funziona

Il freelancer è accessibile. Nessun contratto lungo, nessun onboarding complesso, nessuna struttura burocratica. Per interventi puntuali e ben definiti — un bug critico, una funzionalità semplice, un'integrazione limitata — può essere la risposta giusta. La comunicazione è diretta, senza filtri.

Cosa non funziona

Nessuna continuità. Ogni volta che richiami il freelancer dopo una pausa, deve rientrare nel codice. Deve rileggere, capire come funziona, ricostruire il contesto. Questo ha un costo in tempo ogni volta — e il costo è tuo, non suo.

Incentivi disallineati. Un freelancer pagato a ore o a feature non ha incentivo a costruire architettura pulita, scrivere test, o documentare. Ha incentivo a consegnare velocemente. Nel breve termine va bene. Nel lungo termine accumula debito tecnico che pagherai dopo.

Nessuna governance. Non c'è un processo. Non ci sono demo regolari, sprint, report. Le richieste arrivano quando arrivano, vengono gestite quando vengono gestite. Non c'è prevedibilità.

Disponibilità non garantita. Se il freelancer trova un progetto più interessante o urgente, la tua richiesta aspetta. In un momento critico, questo può essere un problema serio.

Quando ha senso

Ha senso per interventi davvero puntuali e isolati, con scope molto ben definito. Non per la gestione continuativa di un prodotto in evoluzione.

4. Scenario C: La software house

La software house porta struttura, team, e capacità produttiva. Per progetti di una certa complessità è spesso la scelta obbligata. Ma ha caratteristiche che vale la pena conoscere prima di firmare.

Cosa funziona

Una software house ha un team: non sei dipendente da una persona sola. Se qualcuno è assente, c'è qualcun altro. Ci sono processi definiti, project manager, procedure. Per progetti grandi con molte parti in parallelo, questa struttura è un valore reale. Le software house strutturate hanno anche esperienza accumulata su più progetti — architetture viste, problemi risolti, pattern conosciuti.

Cosa non funziona

Il senior ti vende, il junior ti costruisce. Nella grande maggioranza dei casi, il professionista esperto che ti presenta il progetto e ti ispira fiducia non è quello che scrive il tuo codice ogni giorno. Ti viene assegnato chi è disponibile. Questo non è un problema etico — è il modello di business. Ma vale la pena saperlo prima.

La comunicazione è mediata. C'è quasi sempre un project manager o un account manager tra te e il team che sviluppa. Questo introduce latenza, interpretazioni, e possibili distorsioni. Le decisioni tecnicetrate attraverso più livelli prima di diventare codice.

La priorità dipende dal portafoglio clienti. La tua urgenza viene pesata insieme a quella di tutti gli altri clienti della software house. In un momento critico per te, potresti non essere il cliente più importante per loro.

Il costo della struttura. Una software house con project manager, ufficio, commerciali, e overhead organizzativo ha costi che vengono trasferiti sul cliente. Non è un problema — ma significa che paghi anche per la struttura, non solo per il lavoro fatto sul tuo progetto.

Quando ha senso

Ha senso per progetti di alta complessità tecnica, con timeline rigida, che richiedono team di quattro o più persone in parallelo. O quando hai necessità di certificazioni, garanzie contrattuali formali, o rapporti con sistemi enterprise che richiedono struttura.

5. Scenario D: Il team Factory in retainer

Il modello factory — un team dedicato a retainer mensile — è la risposta al problema specifico delle aziende che hanno un prodotto digitale esistente e vogliono farlo evolvere in modo costante, prevedibile, e qualitativo.

Non è adatto a tutti. Ma per chi lo è, risolve i problemi strutturali degli altri tre modelli in modo preciso.

Come funziona

Un retainer mensile significa che ogni mese hai una capacità di sviluppo riservata per te — ore che non vengono vendute ad altri, con lo stesso team che conosce già il tuo sistema.

Il mese tipico è strutturato in tre momenti:

Planning — all'inizio del mese, definisci insieme al team cosa è prioritario. Non una lista infinita di desiderata — un set realistico di obiettivi per le settimane che seguono. Il team contribuisce con stime realistiche e con segnalazioni tecniche (debito da risolvere, rischi emergenti, opportunità di ottimizzazione).

Sprint bisettimanali — il lavoro si svolge in cicli di due settimane con una demo alla fine. Ogni due settimane vedi

qualcosa di funzionante, fornisci feedback, il team integra le correzioni. Non aspetti mesi per scoprire problemi.

Report mensile — alla fine del mese, il team ti consegna un report sintetico: cosa è stato fatto, metriche di qualità (coverage dei test, performance, errori rilevati), e proposte per il mese successivo. Non è burocrazia — è il fondamento della responsabilità reciproca.

Cosa funziona

Continuità di contesto. Lo stesso team, ogni mese. Conosce il tuo codice, il tuo business, le tue priorità, la tua storia tecnica. Non c'è overhead di "rientrare nel codice" ogni volta. Le decisioni sono più rapide e più informate.

Qualità costante. Un factory partner serio incorpora code review, test automatici, deployment automatizzato come standard di lavoro — non come extra. Il -98% di bug in produzione che osserviamo nei team che adottano questo approccio non è casuale: è il risultato di pratiche di qualità integrate nel processo, non aggiunte alla fine.

Governance inclusa. Non stai comprando ore di sviluppo. Stai comprando un rapporto: planning, demo, report, ownership tecnica. Il factory partner è responsabile non solo di scrivere codice, ma di prendere decisioni architetturali sensate e di comunicarle.

Prevedibilità. Sai ogni mese cosa entra e cosa esce. Non hai sorprese sui costi, non hai variazioni imprevedibili di disponibilità, non hai forniture che si interrompono senza preavviso.

Flessibilità nel tempo. Ogni tre mesi puoi rinegoziare la capacità in base alle esigenze del momento. Se hai un lancio importante e hai bisogno di più lavoro, scale up. Se hai un periodo tranquillo, scale down.

Cosa non funziona

Non è adatto a tutto. Se hai bisogno di un'unica modifica all'anno, un progetto a corpo è più sensato. Se hai già un team interno forte e cerchi solo overflow temporaneo, il retainer ha troppa struttura per quello che ti serve. Se non hai ancora un sistema in produzione da far evolvere, prima costruiscilo con un progetto, poi passa alla factory.

Richiede un certo volume di lavoro continuativo. Il modello funziona quando c'è abbastanza da fare ogni mese per giustificare la capacità riservata. Se il tuo software è stabile e non richiede evoluzione, stai pagando per capacità che non usi.

Richiede coinvolgimento da parte tua. Il factory partner non può stabilire le priorità al posto tuo. Ha bisogno di un interlocutore aziendale che partecipa al planning, valuta le demo, e prende decisioni. Il retainer non funziona se dall'altra parte c'è assenza.

****Insight 108 Vision**** — Il modello factory ha portato i team che lo adottano ad aumentare la frequenza di rilascio di quattro volte rispetto al modello a progetto. Non perché lavorino più velocemente — ma perché il ciclo demo-feedback-correzione è strutturato, e i problemi vengono risolti prima di accumularsi.

6. Come scegliere il modello giusto per te

Questa decisione non ha una risposta universale. Ha però dei criteri precisi.

Scegli il developer interno se:

- Il software è il core del tuo business, non uno strumento di supporto
- Hai lavoro sufficiente per giustificare un impegno full-time (non solo quando serve)
- Sei disposto a gestire una persona, con tutti i costi e le responsabilità che ne derivano
- Accetti che uno solo non è mai sufficiente — e sei disposto a costruire un team nel tempo

Scegli il freelancer se:

- Hai bisogno di un intervento puntuale, ben definito, non ricorrente
- Il tuo software non richiede evoluzione continua
- Puoi definire lo scope in modo preciso e autosufficiente

Scegli la software house se:

- Hai un progetto grande, con più parti in parallelo, complessità tecnica elevata
- Hai bisogno di garanzie contrattuali formali o di struttura certificata
- Hai un team interno che gestirà poi il prodotto, e cerchi qualcuno per costruirlo

Scegli il modello factory se:

- Hai un prodotto in produzione che deve evolvere ogni mese
- Hai due-cinque richieste ricorrenti al mese
- Non vuoi o non puoi assumere full-time
- Vuoi qualità costante senza gestire persone
- Vuoi un interlocutore unico con ownership tecnica

Segnale di allerta: se stai usando un freelancer per coprire un bisogno continuo, stai pagando il prezzo della flessibilità senza ottenere nessuno dei benefici della continuità. Vale la pena confrontare il costo totale — ore di freelancer, overhead di onboarding, costo dei bug che emergono dall'assenza di process — con quello di un retainer strutturato.

7. Come valutare un factory partner

Hai deciso che il modello factory fa per te. Come scegli il partner giusto?

Segnali positivi

Ti propone un assessment iniziale del tuo sistema prima di parlare di impegno.

Un buon factory partner vuole capire cosa eredita prima di impegnarsi. Un assessment iniziale — anche breve — produce una valutazione dello stato del codice, delle aree di rischio, del debito tecnico accumulato. Se salti questa fase, entri in una relazione senza sapere davvero cosa stai comprando.

Ha un processo visibile e strutturato.

Sprint, demo, report: non come slogan, ma come pratica reale. Chiediti: mi mostra come ha lavorato con altri clienti? Posso vedere un esempio di report mensile? Sa spiegare come gestisce le priorità quando cambiano a metà mese?

Ti dà accesso a tutto dall'inizio.

Repository, board di lavoro, metriche di qualità. Nessuna scatola nera. Un factory partner che nasconde il lavoro non ha fiducia in quel lavoro.

Il senior che ti parla è quello che scrive il codice.

Uno dei problemi principali della software house è la distanza tra chi vende e chi costruisce. In una factory di qualità, la persona che ti presenta il progetto è quella che ha ownership tecnica reale. Non un commerciale, non un PM — un architetto o un senior developer che risponde direttamente di quello che fa.

Ha un contratto con clausola di uscita chiara.

Un rapporto sano non ha bisogno di blindare il cliente. Se il partner che stai valutando rende difficile l'uscita — clausole lunghe, penali eccessive, difficoltà di trasferimento del codice — stai costruendo una dipendenza, non una partnership.

Segnali di allarme

"Chiamaci quando hai bisogno."

Non è una factory — è un freelancer con un nome diverso. L'assenza di processo è l'assenza di prevedibilità.

Non ti mostra il codice che produce.

Se non puoi vedere il lavoro, non puoi valutarlo. Un partner che non ti dà accesso al repository sta proteggendo qualcosa.

Cambia sviluppatore ogni mese.

La continuità di contesto è il valore principale del modello factory. Se il team ruota, stai pagando per la struttura senza ottenerne il beneficio principale.

Non propone governance o decisioni architetturali.

Se la proposta è "ti facciamo le feature che chiedi", ti stanno vendendo ore di esecuzione — non ownership tecnica. La governance include la capacità di dire "questa feature non si fa così" e spiegare perché, di segnalare rischi tecnici prima che diventino problemi, e di proporre alternative migliori quando la direzione scelta non è quella giusta.

Prezzo solo a ore.

Il prezzo a ore crea un incentivo sbagliato: più tempo ci vuole, più guadagna il fornitore. Un factory partner serio lavora a capacità riservata mensile, con obiettivi definiti — non a ore di presenza.

8. Strutturare la relazione nel tempo

Una factory funziona quando la relazione è strutturata. Questo non è burocratico — è il fondamento della responsabilità reciproca.

Il mese 0: setup e assessment

Prima di iniziare il lavoro, il factory partner analizza il sistema che eredita. Questo produce:

- Una mappa dello stato attuale: cosa funziona bene, cosa è a rischio, cosa ha debito tecnico da gestire
- Una lista delle priorità per il primo mese, concordata con te
- Setup degli strumenti di comunicazione e collaborazione: accesso al repository, board di lavoro, canale di comunicazione diretto
- Accordo sulle modalità operative: come arrivano le richieste, come vengono priorizzate, come vengono comunicate le stime

Non saltare questa fase per iniziare a sviluppare prima. Una settimana investita nel setup vale mesi di lavoro più efficace.

Il regime mensile

Ogni mese ha una struttura ripetibile:

Planning (inizio mese, una-due ore). Definizione delle priorità per il mese. Il team porta la prospettiva tecnica — stime realistiche, segnalazioni di debito tecnico rilevante, rischi emergenti. Tu porti la prospettiva di business — cosa è più urgente, cosa genera più valore, cosa è bloccante per qualcosa d'altro. La combinazione produce una lista di lavoro realistica.

Sprint bisettimanali (ogni due settimane). Demo di quello che è stato fatto. Non una presentazione — il software vivo, testabile. Feedback immediato. Aggiustamenti per lo sprint successivo. Se qualcosa non va nella direzione giusta, lo scopri in due settimane, non in due mesi.

Report mensile (fine mese). Cosa è stato completato, metriche di qualità, note tecniche rilevanti, proposta per il mese successivo. Questo documento è la memoria del progetto — nel tempo produce una storia di come il sistema è evoluto e perché.

La review trimestrale

Ogni tre mesi, una conversazione più ampia: la direzione strategica del prodotto, la soddisfazione reciproca, la necessità di cambiare la capacità mensile. Non è un rinnovo automatico — è una revisione cosciente della relazione.

Questa cadenza previene due problemi comuni: il factory partner che continua a fare lavoro che ha smesso di essere rilevante, e il cliente che rimane in un rapporto che non soddisfa più per inerzia.

*****Insight 108 Vision**** — La soddisfazione del team che lavora con il modello factory è mediamente superiore del cinquanta per cento rispetto ai team che lavorano su progetti a corpo. Non perché il lavoro sia più facile — ma perché la continuità di contesto e la struttura del processo riducono il caos, e il caos è la principale causa di insoddisfazione nel lavoro tecnico.*

9. Cosa aspettarsi nei primi tre mesi

La maggior parte delle relazioni factory attraversa fasi prevedibili. Conoscerle in anticipo evita di trarre conclusioni sbagliate nei momenti di adattamento.

Mese 1 — Esplorazione.

Il team impara il sistema, il tuo business, il tuo modo di lavorare. Il ritmo di produzione è più lento del previsto — non perché il team sia lento, ma perché ogni decisione richiede più contesto. Aspettati domande frequenti, e considera ogni domanda un investimento nel mese due.

Mese 2 — Stabilizzazione.

Il team conosce abbastanza per lavorare in modo più autonomo. La velocità aumenta. Emergono le prime inefficienze del processo — punti di attrito nella comunicazione, aspettative non allineate su come arrivano le richieste, ambiguità nella prioritizzazione. Questo è normale e prezioso: è il momento di allinearsi esplicitamente su come lavorare insieme.

Mese 3 — Regime.

Il flusso di lavoro è fluido. Le stime sono più accurate perché il team conosce il sistema. Le decisioni sono più rapide perché il contesto è condiviso. Il report mensile inizia a essere uno strumento reale, non solo un adempimento.

Se la relazione non ha raggiunto questo regime entro il terzo mese, c'è un problema da affrontare esplicitamente — di processo, di allineamento, o di competenza. Non aspettare il sesto mese sperando che si risolva da solo.

10. Misurare il valore nel tempo

Una relazione factory funziona solo se produce valore misurabile. Non basta la sensazione soggettiva che "le cose vanno meglio." Hai bisogno di indicatori concreti che ti permettano di capire se stai ottenendo quello che hai pagato.

Checklist di indicatori da monitorare:

- [] Quante feature o miglioramenti vengono consegnati al mese, in media?
- [] Quanto tempo passa tra la richiesta e la disponibilità in produzione (tempo di ciclo)?

- ☐ Quanti bug emergono dopo ogni rilascio in produzione?
- ☐ Quante emergenze "bloccanti" si sono verificate negli ultimi tre mesi?
- ☐ La capacità mensile è stata utilizzata al livello concordato?
- ☐ Il debito tecnico segnalato nel mese 0 è stato ridotto?
- ☐ I tempi stimati corrispondono ai tempi reali (accuratezza delle stime)?

Questi numeri non sono perfetti — nessun indicatore lo è. Ma insieme descrivono la salute della relazione in modo più affidabile di qualsiasi valutazione soggettiva.

Condividi questi indicatori con il factory partner. Un partner serio li monitora già internamente — e li mostra proattivamente nel report mensile.

Segnale di allarme: se il tuo factory partner non ha questi numeri o non vuole condividerli, stai pagando per un servizio che non si misura. Ogni servizio che non si misura non si può migliorare.

Vuoi andare oltre?

Vuoi applicare questo metodo alla tua azienda? Prenota 30 minuti con noi su 108vision.it — gratuito, senza impegno.

108 Vision — Costruiamo la direzione, non solo il codice.